

MINISTRY OF EDUCATION AND RESEARCH



TECHNICAL UNIVERSITY
OF CLUJ-NAPOCA, ROMANIA

Assignment 3

Software Design

Author: Andrei Varga

Group: 30433

FACULTY OF AUTOMATION AND COMPUTER SCIENCE

May 2026

Contents

1	Problem definition	2
2	Analysis phase – Use cases & requirements	2
2.1	Functional requirements	2
2.2	Use cases	2
3	Design phase	3
3.1	Architecture overview	3
3.2	Entity-relationship diagram	4
3.3	Service descriptions	4
3.4	Design patterns	5
3.5	Class diagrams	5
3.6	Sequence diagrams	6
4	Implementation phase – Application	6
4.1	Tools and technologies	6
4.2	Application structure	7
4.3	User manual	7
5	Testing and validation	7

1 Problem definition

This assignment extends the online marketplace from Assignments 1 and 2 by migrating to a microservices architecture. The domain is unchanged: shops sell products, users place orders. Each service is independently deployable with its own database. The system also introduces CQRS, asynchronous messaging via RabbitMQ, and two design patterns: Template Method and Command.

Section 2 covers requirements and use cases. Section 3 covers design. Section 4 covers implementation. Section 5 covers testing.

Deliverables: a GitHub branch with all five microservices and the React frontend, and this documentation.

2 Analysis phase – Use cases & requirements

2.1 Functional requirements

- **FR-01** Visitors can browse products and shops without authentication.
- **FR-02** Users can register and log in; authentication returns a JWT token.
- **FR-03** Administrators can perform full CRUD on users, products, shops and orders, and can promote users to Store Manager.
- **FR-04** Store Managers can manage stock for their assigned shop (add, remove, increment, decrement quantities).
- **FR-05** Authenticated users can view their orders and create new ones from in-stock products.
- **FR-06** Products can be filtered by price range and sorted by name or price.
- **FR-07** The product list can be exported in JSON, XML or CSV format, respecting active filters and sorting.
- **FR-08** The system publishes domain events (product created/updated) to RabbitMQ; the notifications service consumes them and logs to MongoDB.
- **FR-09** If an email address is provided on a product write operation, the notifications service sends an email.
- **FR-10** Each service implements CQRS with separate command and query controllers and services.

2.2 Use cases

The system has four actor types: Visitor (unauthenticated), User, Store Manager and Administrator. The Notifications Service acts as an internal actor.

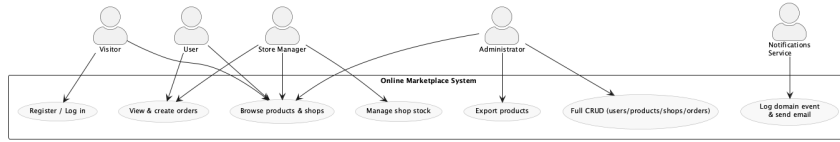


Figure 1: Use case diagram

Browse products and shops: Any visitor can view the landing page which lists all products and shops, with optional price filtering.

Register and log in: A visitor submits credentials; on success a JWT is issued and stored in the browser, and the user is redirected to their role-specific dashboard.

Create a product: An administrator fills the product form and optionally provides an email. The product is saved and a `PRODUCT_CREATED` event is published to RabbitMQ.

Export products: An administrator clicks an export button (JSON/XML/CSV); the current filtered and sorted product list is returned as a downloadable file.

Manage shop stock: A store manager navigates to their shop page and adds, removes or adjusts quantities of products in stock.

Create an order: A user or store manager selects quantities from in-stock products and submits; the order is linked to their user ID and the total is calculated automatically.

Domain event notification: The notifications service consumes a `DomainEvent` from RabbitMQ, saves it to MongoDB, and sends an email if one was provided.

3 Design phase

3.1 Architecture overview

The system consists of five Spring Boot microservices, each with its own database. The React frontend communicates with each service directly over REST. The products-service additionally publishes domain events to RabbitMQ, which the notifications-service consumes asynchronously.

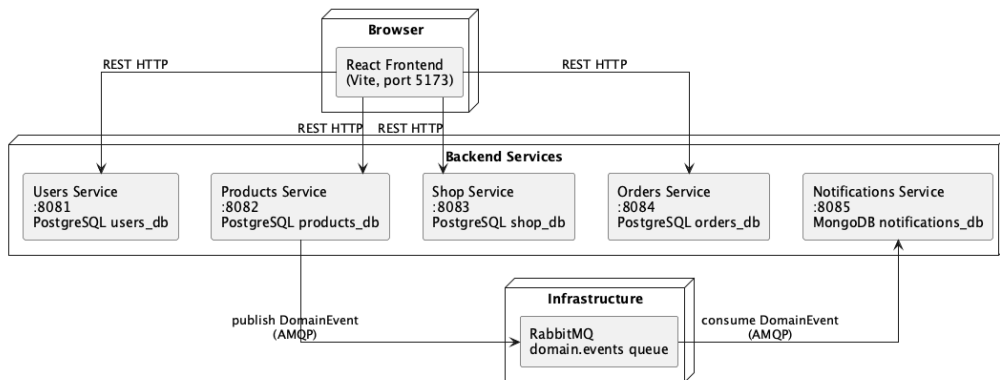


Figure 2: High-level architecture diagram

3.2 Entity-relationship diagram

Each service owns its own schema. Cross-service references (e.g. shopId in products, userId in orders) are logical only and not enforced at the database level.

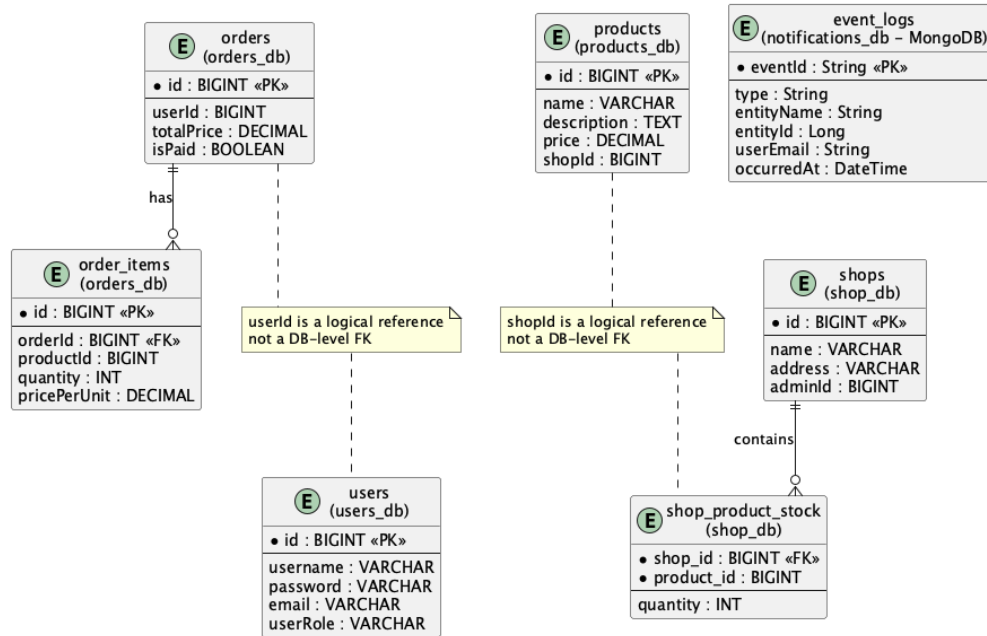


Figure 3: Entity-relationship diagram

- **users** (users-service): id, username, password (BCrypt), email, userRole.
- **products** (products-service): id, name, description, price, shopId.
- **shops** (shop-service): id, name, address, adminId; stock stored as a map of productId to quantity in a join table.
- **orders / order_items** (orders-service): order has userId, totalPrice, isPaid; items have productId, quantity, pricePerUnit.
- **event_logs** (notifications-service, MongoDB): eventId, type, entityId, userEmail, occurredAt.

3.3 Service descriptions

Users Service (port 8081): handles registration, JWT login, user CRUD and role management. Spring Security secures write endpoints; reads are public. CQRS is applied with separate UserCommandService and UserQueryService.

Products Service (port 8082): manages the product catalogue. Implements the Template Method pattern for export and publishes a DomainEvent to RabbitMQ after every write.

Shop Service (port 8083): manages shops and their product stock map. Exposes dedicated endpoints for stock increments, decrements and removal.

Orders Service (port 8084): manages orders and order items. Recalculates total price on item changes. Supports filtering by user and paid status.

Notifications Service (port 8085): listens on the RabbitMQ queue, saves every event to MongoDB and optionally sends an email via JavaMail. A custom ClassMapper is used to bypass the TypeId AMQP header mismatch between services.

3.4 Design patterns

Template Method (Behavioral)

Problem: exporting products in JSON, XML and CSV follows the same algorithm (header, body, footer, assemble) but with format-specific logic.

Solution: abstract class ProductExporter defines the template method export(). Concrete subclasses JsonProductExporter, CsvProductExporter and XmlProductExporter override only the format-specific steps.

Command (Behavioral)

Problem: UI actions (create, update price, update description, delete) need to be decoupled from event handlers.

Solution: a Command interface with execute() is defined in TypeScript. Concrete command classes (CreateProductCommand, UpdatePriceCommand, etc.) encapsulate the API call and a success callback. ProductsPage instantiates and calls the appropriate command on user interaction.

3.5 Class diagrams

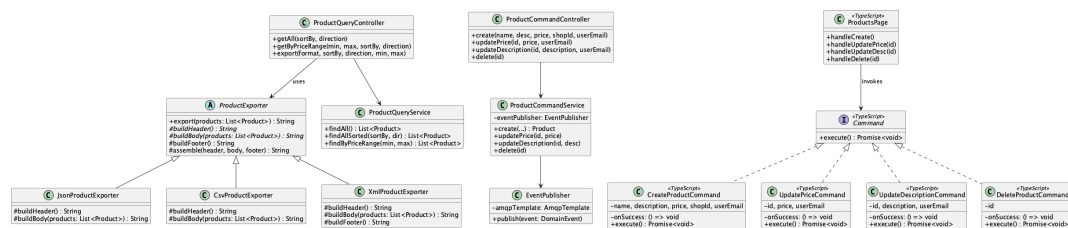


Figure 4: Class diagram – products-service and design patterns

3.6 Sequence diagrams

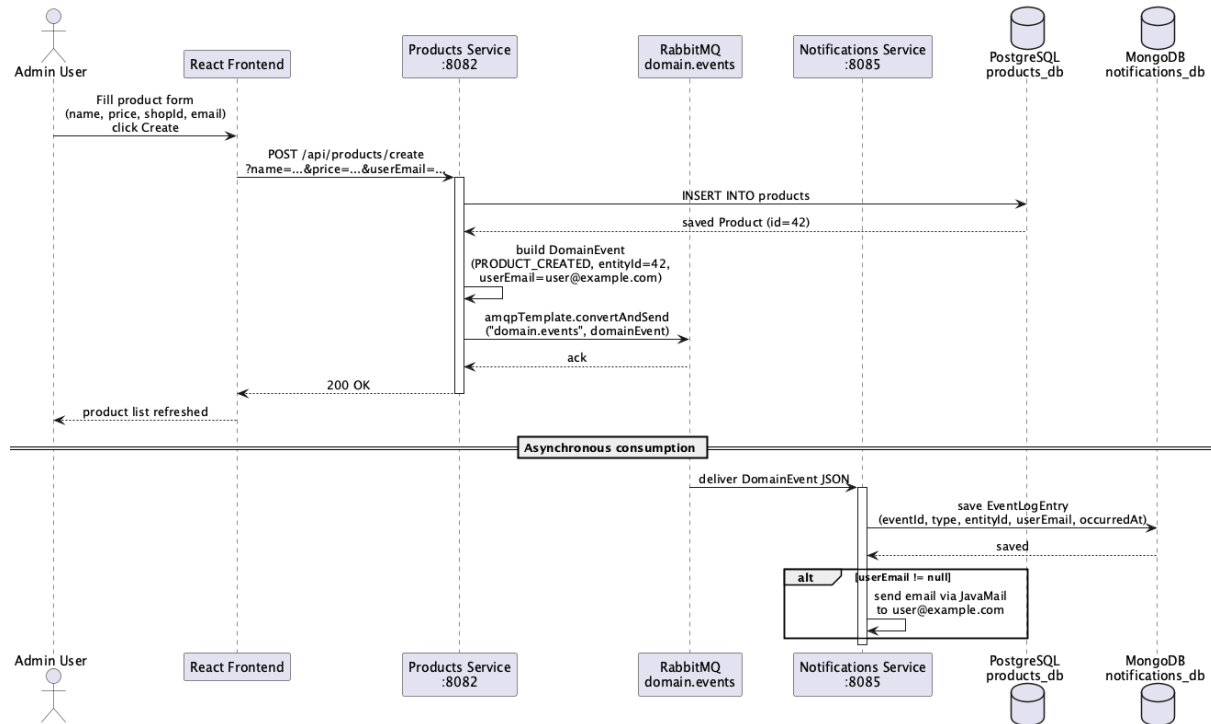


Figure 5: Sequence diagram – product creation and notification flow

4 Implementation phase – Application

4.1 Tools and technologies

- **Java 21 / Spring Boot 3.x** [walls2022spring]: REST controllers (Spring MVC), JPA/Hibernate, Spring Security (JWT), Spring AMQP.
- **PostgreSQL**: relational database for users, products, shops and orders services.
- **MongoDB**: document store for the notifications service event log.
- **RabbitMQ** [rabbitmq]: asynchronous messaging between products-service and notifications-service.
- **React 18 + Vite + TypeScript** [react]: single-page frontend with React Router and Axios.
- **Lombok / Jackson / JavaMail**: boilerplate reduction, JSON serialisation, email sending.
- **Maven / Git**: build and version control.

4.2 Application structure

Each service follows the same internal structure: config, controller (command + query), service (command + query), repository, model, dto. The products-service also has a messaging and export package. The notifications-service has a messaging and service package.

The frontend is organised as: api (one file per service), commands (Command interface and product commands), components (Navbar, ProtectedRoute), pages (one per route), utils/auth.ts.

4.3 User manual

Start PostgreSQL, MongoDB and RabbitMQ, then each Spring Boot service. Start the frontend:

```
cd frontend && npm install && npm run dev
```

Navigate to <http://localhost:5173>.

Visitor: landing page shows all products and shops. Login button redirects to the login form, which also has a register option.

Administrator: full access to Users, Products, Shops and Orders pages. On the Products page, export buttons (JSON/CSV/XML) download the currently filtered and sorted list. A notification email field triggers email alerts on product changes.

Store Manager: access to My Shop (stock management), My Orders and Shops pages.

User: access to My Orders page, which lists in-stock products for order creation.

5 Testing and validation

Each service has JUnit 5 unit tests using Mockito to mock repositories. Tests cover both command and query operations.

- **users-service:** register with BCrypt encoding, role update, delete, findAll, findByRole, findByUsername.
- **products-service:** create/updatePrice/updateDescription with EventPublisher invocation; findAll and findByPriceRange; all three exporters produce correct output.
- **shop-service:** stock increment, decrement, add, remove; findAll and findByAdminId.
- **orders-service:** order creation, item add/remove, markAsPaid, total price recalculation; filter by userId and isPaid.

Tests are run with `mvn test` in each service directory.